



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Problem Set 4 — Applied Problems

Guía de Resolución con Python

**Capítulo: Soluciones Numéricas de Ecuaciones
Diferenciales Ordinarias**

Carrera:

Ingeniería Mecatrónica — Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Docente:

Ing. Luis Jesús Padrón Padrón

Fecha:

18 de Mayo de 2026

Contents

Introducción	2
1 Problema 1 — Taylor de Orden 2: $y' = x^2 - 4y$	2
2 Problema 3 — Taylor de Orden 2: $y' = \sin y$	3
3 Problema 5 — Conversión a Sistemas de Primer Orden	4
4 Problema 7 — Periodo del Péndulo No Lineal	6
5 Problema 9 — Masa-Resorte con Fuerza Variable	7
6 Problema 11 — Péndulo con Soporte Oscilante	8
7 Problema 13 — Tiro Parabólico con Arrastre Aerodinámico	9
8 Problema 15 — Péndulo Elástico	10
9 Problema 17 — Masa-Resorte con Fricción de Coulomb	12
10 Problema 19 — Función de Bessel $J_0(x)$	13
11 Problema 21 — Circuito RLC: Sistema Rígido	14
Resumen de Bibliotecas y Métodos	16

Introducción

Este documento resuelve los problemas aplicados (*Applied Problems and Projects*) del capítulo dedicado a la **Solución Numérica de Ecuaciones Diferenciales Ordinarias**, utilizando Python junto con las bibliotecas `numpy`, `scipy.integrate` y `matplotlib`. Cada sección incluye: una descripción del problema, la estrategia numérica empleada, el código fuente comentado y la salida de terminal correspondiente.

Contexto Teórico: Métodos para EDOs

Una ecuación diferencial ordinaria (EDO) de primer orden tiene la forma general:

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0$$

Los métodos numéricos buscan aproximar $y(x)$ en una secuencia discreta de puntos $x_0, x_1, x_2, \dots$ separados por un paso h . Las principales familias son:

Table 1: Comparativa de métodos numéricos para EDOs

Método	Orden de error	Evaluaciones	Aplicación
Euler	$O(h^2)$ local	1 por paso	Estimaciones rápidas
Taylor orden 2	$O(h^3)$ local	2 (requiere y'')	Cuando f es derivable
RK4	$O(h^5)$ local	4 por paso	Estándar general
RK45 (Dormand-Prince)	Adaptativo	Variable	Paso adaptativo
BDF	Implícito	Variable	Sistemas rígidos (<i>stiff</i>)

Conversión a sistemas: Cualquier EDO de orden n puede reescribirse como un sistema de n EDOs de primer orden mediante el cambio $y_1 = y$, $y_2 = y'$, $\dots$, $y_n = y^{(n-1)}$. Esto permite que `scipy.integrate.solve_ivp` resuelva problemas de cualquier orden.

Eventos: La opción `events` de `solve_ivp` detecta cruces por cero de una función monitor, permitiendo encontrar el instante exacto en que ocurre un fenómeno físico (impacto, paso por el equilibrio, etc.).

1 Problema 1 — Taylor de Orden 2: $y' = x^2 - 4y$

Enunciado

Aproximar $y(0.1)$ para la EDO $y' = x^2 - 4y$ con $y(0) = 1$, usando el método de Taylor de orden 2 con $h = 0.05$ (dos pasos).

Solución Analítica del Método

Las derivadas necesarias son:

$$y' = x^2 - 4y \quad (1)$$

$$y'' = 2x - 4y' \quad (2)$$

La fórmula recursiva es:

$$y_{n+1} = y_n + h y'_n + \frac{h^2}{2} y''_n$$

Listing 1: Método de Taylor de orden 2 para $y' = x^2 - 4y$

```

1 # Datos iniciales
2 x0, y0 = 0.0, 1.0
3 x_final = 0.1
4 n_pasos = 2
5 h = (x_final - x0) / n_pasos
6
7 def y_prima(x, y):
8     return x**2 - 4 * y
9
10 def y_biprima(x, y_p):
11     return 2 * x - 4 * y_p
12
13 x, y = x0, y0
14 for i in range(1, n_pasos + 1):
15     yp = y_prima(x, y)
16     ypp = y_biprima(x, yp)
17     # Fórmula de Taylor orden 2
18     y_nueva = y + h * yp + ((h**2) / 2) * ypp
19     x_nueva = x + h
20     print(f"Paso {i}: x = {x_nueva:.2f}, y = {y_nueva:.7f}")
21     x, y = x_nueva, y_nueva

```

Terminal Output

```

¡Arrancamos la iteración, cracks!
Paso 1: x = 0.05, y = 0.8200000 Paso 2: x = 0.10, y = 0.6726375
¡Listo, papillos! El valor estimado de y(0.1) es: 0.6726375

```

Verificación

La solución exacta es $y(x) = \frac{x^2}{4} - \frac{x}{8} + \frac{1}{32} + \frac{31}{32}e^{-4x}$, que evaluada en $x = 0.1$ da $y(0.1) \approx 0.6717$. El error con dos pasos de Taylor-2 es $\approx 9 \times 10^{-4}$, consistente con el orden global $O(h^2)$.

2 Problema 3 — Taylor de Orden 2: $y' = \sin y$

Enunciado

Aproximar $y(0.5)$ para la EDO $y' = \sin y$ con $y(0) = 1$, usando Taylor de orden 2 con paso $h = 0.1$ (cinco pasos).

Derivadas

Como f no depende explícitamente de x :

$$y' = \sin y \quad (3)$$

$$y'' = \cos(y) \cdot y' = \cos(y) \sin(y) = \frac{1}{2} \sin(2y) \quad (4)$$

Listing 2: Taylor de orden 2 para $y' = \sin y$

```

1 import math
2
3 x0, y0 = 0.0, 1.0
4 x_final = 0.5
5 h = 0.1
6 n_pasos = int(round((x_final - x0) / h))
7
8 def y_prima(y):
9     return math.sin(y)
10
11 def y_biprima(y, y_p):
12     return math.cos(y) * y_p
13
14 x, y = x0, y0
15 print(f"Paso 0: x = {x:.1f}, y = {y:.7f}")
16 for i in range(1, n_pasos + 1):
17     yp = y_prima(y)
18     ypp = y_biprima(y, yp)
19     y_nueva = y + h * yp + ((h**2) / 2) * ypp
20     x_nueva = x + h
21     print(f"Paso {i}: x = {x_nueva:.1f}, y = {y_nueva:.7f}")
22     x, y = x_nueva, y_nueva

```

Terminal Output

```

¡Arrancamos la iteración, cracks!
  Paso 0: x = 0.0, y = 1.0000000 Paso 1: x = 0.1, y = 1.0865781 Paso 2: x = 0.2, y
= 1.1700076 Paso 3: x = 0.3, y = 1.2493829 Paso 4: x = 0.4, y = 1.3240029 Paso 5: x
= 0.5, y = 1.3933280
  ¡Listo el pollo! El valor estimado de y(0.5) es: 1.3933280

```

Observación

A medida que y crece hacia $\pi/2$, $\sin y$ se acerca a 1 y la pendiente se vuelve máxima. Para valores de x mayores, y tendería asintóticamente a π , donde $\sin y \rightarrow 0$ y el sistema alcanza su equilibrio estable.

3 Problema 5 — Conversión a Sistemas de Primer Orden

Enunciado

Reescribir como sistemas de primer orden, listos para ser resueltos numéricamente, las siguientes EDOs:

- (a) $\ln y' + y = \sin x$
- (b) $y''y - xy' - 2y^2 = 0$
- (c) $y^{(4)} - 4y''\sqrt{1 - y^2} = 0$
- (d) $(y'')^2 = |32y'x - y^2|$

Estrategia

Para una EDO de orden n se define el vector de estado $\mathbf{Y} = [y_1, y_2, \dots, y_n]^T$ donde $y_k = y^{(k-1)}$. La derivada del vector se obtiene de la EDO despejando la derivada de orden más alto.

Table 2: Transformación a sistemas de primer orden

Eq.	Sistema	Tamaño
(a)	$y'_1 = e^{\sin x - y_1}$	1 ecuación
(b)	$y'_1 = y_2; \quad y'_2 = (xy_2 + 2y_1^2)/y_1$	2 ecuaciones
(c)	$y'_k = y_{k+1} \ (k = 1, 2, 3); \quad y'_4 = 4y_3\sqrt{1 - y_1^2}$	4 ecuaciones
(d)	$y'_1 = y_2; \quad y'_2 = \pm\sqrt{ 32y_2x - y_1^2 }$	2 ecuaciones (dos ramas)

Listing 3: Funciones de sistema de primer orden

```

1 import numpy as np
2
3 # (a) y' = e^(sin x - y)
4 def ecuacion_a(x, Y):
5     y1 = Y[0]
6     dy1_dx = np.exp(np.sin(x) - y1)
7     return [dy1_dx]
8
9 # (b) y1' = y2; y2' = (x*y2 + 2*y1^2) / y1
10 def ecuacion_b(x, Y):
11     y1, y2 = Y
12     dy1_dx = y2
13     dy2_dx = (x * y2 + 2 * y1**2) / y1
14     return [dy1_dx, dy2_dx]
15
16 # (c) y1'..y3' = cadena; y4' = 4*y3*sqrt(1 - y1^2)
17 def ecuacion_c(x, Y):
18     y1, y2, y3, y4 = Y
19     dy1_dx = y2
20     dy2_dx = y3
21     dy3_dx = y4
22     dy4_dx = 4 * y3 * np.sqrt(1 - y1**2)
23     return [dy1_dx, dy2_dx, dy3_dx, dy4_dx]
24
25 # (d) y2' = +sqrt(|32*y2*x - y1^2|) (rama positiva)
26 def ecuacion_d_positiva(x, Y):
27     y1, y2 = Y

```

```

28     dy1_dx = y2
29     dy2_dx = np.sqrt(np.abs(32 * y2 * x - y1**2))
30     return [dy1_dx, dy2_dx]

```

Terminal Output

```

¡Todo listo, papillos! Las funciones están cargadas. Para usarlas con solve_ivp, solo pásalas como argumento.
1, y = [2, 3] : dy/dx = [3.0, 5.5]

```

Advertencias:

- En (b), $y_1 = 0$ produce división entre cero; debe evitarse esa condición inicial.
- En (c), $|y_1| > 1$ vuelve negativo el radicando; la solución sale del dominio real.
- En (d), la rama $\pm$ se elige según la condición inicial de y' .

4 Problema 7 — Periodo del Péndulo No Lineal

Enunciado

La ecuación adimensional del péndulo simple es:

$$\frac{d^2\theta}{d\tau^2} + \sin\theta = 0$$

Con $\theta(0) = 1$ rad y $\theta'(0) = 0$, calcular numéricamente el periodo y compararlo con la aproximación de ángulos pequeños $T_{\text{peq}} = 2\pi$.

Estrategia

Se aprovecha que un péndulo conservativo es simétrico: el tiempo en pasar de θ_{max} a $\theta = 0$ es exactamente $T/4$. Se usa un evento que detecta el cruce por cero del ángulo y se detiene la simulación.

Listing 4: Periodo del péndulo no lineal por eventos

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 theta_inicial = 1.0
5 omega_inicial = 0.0
6 estado_inicial = [theta_inicial, omega_inicial]
7
8 def pendulo_adimensional(tau, estado):
9     theta, omega = estado
10    return [omega, -np.sin(theta)]
11
12 def cruza_el_centro(tau, estado):
13    return estado[0]
14 cruza_el_centro.terminal = True
15 cruza_el_centro.direction = -1

```

```

16
17 solucion = solve_ivp(pendulo_adimensional, (0, 10), estado_inicial,
18                       events=cruza_el_centro, dense_output=True)
19
20 cuarto_de_periodo = solucion.t_events[0][0]
21 periodo_numerico = 4 * cuarto_de_periodo
22 periodo_pequeno = 2 * np.pi
23
24 print(f"Periodo numérico real (1 rad): {periodo_numerico:.5f}")
25 print(f"Periodo aprox. ángulo pequeño: {periodo_pequeno:.5f}")
26 diff = (periodo_numerico - periodo_pequeno) / periodo_pequeno * 100
27 print(f"Diferencia relativa: {diff:.2f}%")

```

Terminal Output

```

¡Listo, máquinas! El periodo numérico real (adimensional) para 1 rad es: 6.53353 El periodo
con la fórmula de ángulos pequeños es: 6.28319
¡Qué locura! El periodo real es un 3.991 radián ya no es un 'ángulo pequeño'.

```

Interpretación Física

La aproximación de ángulos pequeños subestima el periodo en aproximadamente 4% para una amplitud de 1 rad ($\approx 57.3^\circ$). La fórmula exacta involucra una integral elíptica completa de primer tipo:

$$T = 4\sqrt{\frac{L}{g}} K\left(\sin \frac{\theta_{\max}}{2}\right)$$

y la corrección de primer orden es $T \approx T_0(1 + \theta_{\max}^2/16)$.

5 Problema 9 — Masa-Resorte con Fuerza Variable

Enunciado

Un sistema masa-resorte ($m = 2.5$ kg, $k = 75$ N/m) parte del reposo y es excitado por:

$$P(t) = \begin{cases} 10t & 0 \leq t < 2 \\ 20 & t \geq 2 \end{cases}$$

Determinar el desplazamiento máximo y el instante en que ocurre.

Modelo

$$\ddot{y} + \frac{k}{m}y = \frac{P(t)}{m}$$

Con frecuencia natural $\omega_n = \sqrt{k/m} = \sqrt{30} \approx 5.477$ rad/s. La fuerza tipo rampa-escalón es continua pero su derivada presenta un quiebre en $t = 2$ s.

Listing 5: Sistema masa-resorte con fuerza de rampa-escalón

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4
5 m, k = 2.5, 75.0
6
7 def fuerza_P(t):
8     return 10.0 * t if t < 2.0 else 20.0
9
10 def masa_resorte_ode(t, estado):
11     y, y_dot = estado
12     y_ddot = fuerza_P(t) / m - (k / m) * y
13     return [y_dot, y_ddot]
14
15 estado_inicial = [0.0, 0.0]
16 t_span = (0, 5)
17 t_eval = np.linspace(*t_span, 1000)
18
19 solucion = solve_ivp(masa_resorte_ode, t_span, estado_inicial, t_eval=
    t_eval)
20 t, y = solucion.t, solucion.y[0]
21
22 max_y = np.max(y)
23 max_t = t[np.argmax(y)]
24 print(f"Desplazamiento máximo: {max_y:.4f} m en t = {max_t:.4f} s")

```

Terminal Output

```

¡Calculando la trayectoria, papillos...
  ¡Misión cumplida! El desplazamiento MÁXIMO de la masa es:  0.3556 metros. Y ocurre
exactamente en el segundo t = 2.5736 s. ¡Qué locura!

```

Análisis

Durante $0 \leq t < 2$ s, el sistema responde a una rampa lineal y oscila alrededor del desplazamiento estático $y_{\text{est}}(t) = P(t)/k$. Al volverse $P(t)$ constante ($= 20$ N) tras $t = 2$ s, el equilibrio se fija en $y_{\text{eq}} = 20/75 \approx 0.267$ m, y la masa oscila libremente alrededor de ese valor; el primer pico tras el cambio supera a y_{eq} por la energía acumulada.

6 Problema 11 — Péndulo con Soporte Oscilante

Enunciado

Un péndulo de longitud $L = 1$ m está montado en un collarín que oscila verticalmente con amplitud $Y = 0.25$ m y frecuencia $\omega = 2.5$ rad/s. La EDO resultante es:

$$\ddot{\theta} = -\frac{g}{L} \sin \theta + \frac{\omega^2 Y}{L} \cos \theta \sin(\omega t)$$

Determinar el ángulo máximo θ en el intervalo $[0, 10]$ s con condiciones iniciales de reposo.

Listing 6: Péndulo forzado paramétricamente

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4
5 g, L, Y, omega = 9.80665, 1.0, 0.25, 2.5
6
7 def pendulo_ode(t, y):
8     theta, theta_dot = y
9     theta_ddot = -(g/L) * np.sin(theta)
10                  + (omega**2 * Y / L) * np.cos(theta) * np.sin(omega*t)
11                  )
12     return [theta_dot, theta_ddot]
13
14 y0 = [0.0, 0.0]
15 t_eval = np.linspace(0, 10, 1000)
16 solucion = solve_ivp(pendulo_ode, (0, 10), y0, t_eval=t_eval, method='
17                    RK45')
18
19 t, theta = solucion.t, solucion.y[0]
20 max_theta = np.max(np.abs(theta))
21 max_t = t[np.argmax(np.abs(theta))]
22 print(f"|   _max = {max_theta:.4f} rad   en   t = {max_t:.2f} s")

```

Terminal Output

```

¡Listo, papillos! El valor máximo de theta durante el periodo es de 0.3624 radianes. Y
sucede exactamente en el segundo t = 7.82 s. ¡Pasadísimo de lanza!

```

Discusión

La frecuencia natural del péndulo es $\omega_n = \sqrt{g/L} \approx 3.13$ rad/s, mientras que la forzante actúa a 2.5 rad/s. Como ambas son cercanas pero no idénticas, aparece un fenómeno de **batido** con crecimiento gradual de la amplitud. Si ω coincidiera con ω_n , ocurriría resonancia paramétrica y la amplitud divergiría.

7 Problema 13 — Tiro Parabólico con Arrastre Aerodinámico

Enunciado

Una bola de $m = 0.25$ kg se lanza con $v_0 = 50$ m/s formando 30° con la horizontal. La fuerza de arrastre se modela con $F_D = C_D \sqrt{v} \vec{v}$, donde $C_D = 0.03$ kg/(m·s)^{1/2}. Calcular tiempo de vuelo t_v y alcance R .

Sistema de Ecuaciones

$$\ddot{x} = -\frac{C_D}{m} \sqrt{v} \dot{x}, \quad \ddot{y} = -\frac{C_D}{m} \sqrt{v} \dot{y} - g, \quad v = \sqrt{\dot{x}^2 + \dot{y}^2}$$

Se utiliza un **evento** con dirección -1 que detecta cuando y cruza el cero descendiendo.

Listing 7: Proyectil con resistencia del aire vía evento de impacto

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4
5 m, v0, angle = 0.25, 50.0, 30.0
6 C_D, g = 0.03, 9.80665
7 theta = np.radians(angle)
8 estado_inicial = [0.0, 0.0, v0*np.cos(theta), v0*np.sin(theta)]
9
10 def proyectil_ode(t, estado):
11     x, y, vx, vy = estado
12     v = np.sqrt(vx**2 + vy**2)
13     ax = -(C_D/m) * vx * np.sqrt(v)
14     ay = -(C_D/m) * vy * np.sqrt(v) - g
15     return [vx, vy, ax, ay]
16
17 def toca_piso(t, estado):
18     return estado[1]
19 toca_piso.terminal = True
20 toca_piso.direction = -1
21
22 solucion = solve_ivp(proyectil_ode, (0, 10), estado_inicial,
23                       events=toca_piso, dense_output=True)
24
25 tiempo_vuelo = solucion.t_events[0][0]
26 rango_R = solucion.y_events[0][0][0]
27 print(f"Tiempo de vuelo: {tiempo_vuelo:.4f} s")
28 print(f"Rango R: {rango_R:.4f} m")

```

Terminal Output

```

¡Misión cumplida, papillos! El tiempo de vuelo de la bolita es de: 3.6512 s. Y el rango
R (la distancia total) es de: 109.2647 m. ¡Qué locura!

```

Comparación con Tiro Ideal

Sin arrastre, las fórmulas clásicas dan:

$$t_v^{\text{ideal}} = \frac{2v_0 \sin \theta}{g} = 5.10 \text{ s}, \quad R^{\text{ideal}} = \frac{v_0^2 \sin 2\theta}{g} = 220.8 \text{ m}$$

La resistencia del aire reduce el alcance casi a la mitad: este es el motivo por el cual los proyectiles reales no siguen parábolas perfectas, sino curvas asimétricas con descenso más pronunciado.

8 Problema 15 — Péndulo Elástico

Enunciado

Un péndulo elástico consiste en una masa $m = 0.25 \text{ kg}$ suspendida de una cuerda elástica de longitud natural $L = 0.5 \text{ m}$ y rigidez $k = 40 \text{ N/m}$. Se suelta desde el reposo a $\theta_0 = 60^\circ$ con

la cuerda sin estirar. Determinar el instante en que el péndulo pasa por la vertical ($\theta = 0$) por primera vez y la longitud de la cuerda en ese momento.

Ecuaciones de Movimiento (coordenadas polares)

$$\ddot{r} = r\dot{\theta}^2 + g\cos\theta - \frac{k}{m}(r - L) \quad (5)$$

$$\ddot{\theta} = \frac{-2\dot{r}\dot{\theta} - g\sin\theta}{r} \quad (6)$$

Listing 8: Péndulo elástico con evento de cruce por la vertical

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 g, k, L, m = 9.80665, 40.0, 0.5, 0.25
5 theta_0 = np.radians(60)
6 estado_inicial = [L, 0.0, theta_0, 0.0] # [r, r', theta, theta']
7
8 def pendulo_elastico(t, estado):
9     r, r_dot, theta, theta_dot = estado
10    r_ddot = r * theta_dot**2 + g*np.cos(theta) - (k/m)*(r - L)
11    theta_ddot = (-2*r_dot*theta_dot - g*np.sin(theta)) / r
12    return [r_dot, r_ddot, theta_dot, theta_ddot]
13
14 def pasa_por_el_centro(t, estado):
15     return estado[2]
16
17 pasa_por_el_centro.terminal = True
18 pasa_por_el_centro.direction = -1
19
20 solucion = solve_ivp(pendulo_elastico, (0, 5), estado_inicial,
21                      events=pasa_por_el_centro)
22
23 tiempo_cruce = solucion.t_events[0][0]
24 r_final = solucion.y_events[0][0][0]
25 print(f"Cruce por =0 en t = {tiempo_cruce:.4f} s")
26 print(f"Longitud de cuerda en ese instante: r = {r_final:.4f} m")

```

Terminal Output

```

¡Haciendo los cálculos bien perrones, cracks...
  ¡Misión cumplida, papillos! El péndulo cruza el centro por primera vez en t = 0.4031
s. En ese exacto instante, la longitud de la cuerda r es de: 0.5612 metros.

```

Comentarios

El péndulo elástico exhibe acoplamiento entre los modos radial y angular: cuando la masa pasa por la vertical, la cuerda está estirada ≈ 6 cm respecto a su longitud natural debido a la fuerza centrífuga $mr\dot{\theta}^2$ y al peso mg . Este sistema puede mostrar comportamiento caótico para amplitudes y rigideces críticas.

9 Problema 17 — Masa-Resorte con Fricción de Coulomb

Enunciado

Un sistema con $k = 3000$ N/m, $m = 6$ kg y coeficiente de fricción $\mu = 0.5$ se libera desde $y_0 = 0.1$ m. Determinar el primer pico positivo y compararlo con la fórmula analítica:

$$y_{\text{pico}} = y_0 - \frac{4\mu mg}{k}$$

Modelo

$$\ddot{y} = -\frac{k}{m}y - \mu g \operatorname{sgn}(\dot{y})$$

El término $\operatorname{sgn}(\dot{y})$ provoca discontinuidad de la fuerza al invertirse la velocidad; por ello se restringe `max_step = 0.001` para que el integrador detecte fielmente los cambios de signo.

Listing 9: Vibración amortiguada por fricción de Coulomb

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 k, m, mu, g, y0 = 3000.0, 6.0, 0.5, 9.80665, 0.1
5 y_pico_teorico = y0 - (4 * mu * m * g) / k
6
7 def masa_friccion_ode(t, estado):
8     y, y_dot = estado
9     friccion = mu * g * np.sign(y_dot)
10    y_ddot = -(k/m)*y - friccion
11    return [y_dot, y_ddot]
12
13 def cruce_velocidad_cero(t, estado):
14     return estado[1]
15 cruce_velocidad_cero.direction = -1
16 cruce_velocidad_cero.terminal = True
17
18 solucion = solve_ivp(masa_friccion_ode, (0, 2), [y0, 0.0],
19                     events=cruce_velocidad_cero, max_step=0.001,
20                     dense_output=True)
21
22 t_pico_num = solucion.t_events[0][0]
23 y_pico_num = solucion.y_events[0][0][0]
24 print(f"Numérico:    {y_pico_num:.6f} m")
25 print(f"Teórico:     {y_pico_teorico:.6f} m")
26 print(f"Error:        {abs(y_pico_num - y_pico_teorico):.8f} m")

```

Terminal Output

```

¡Haciendo que la compu sude, papillos...
¡Resultados fresquecitos! El pico calculado por Python (Numérico):  0.060778 m El pico
de la fórmula del libro (Teórico):  0.060778 m La diferencia entre ambos es de:  0.00000022
m. ¡Prácticamente idénticos!

```

Interpretación

A diferencia del amortiguamiento viscoso (proporcional a $\dot{y}$), la fricción de Coulomb produce una pérdida **constante** de amplitud por ciclo:

$$\Delta y_{\text{ciclo}} = \frac{4\mu mg}{k}$$

El sistema se detiene cuando $|y| \leq \mu mg/k$ (banda muerta); a diferencia del amortiguamiento viscoso, la posición final no es necesariamente $y = 0$.

10 Problema 19 — Función de Bessel $J_0(x)$

Enunciado

La ecuación de Bessel de orden cero:

$$y'' + \frac{1}{x}y' + y = 0, \quad y(0) = 1, \quad y'(0) = 0$$

tiene como solución regular en el origen la función $J_0(x)$. Calcular $J_0(5)$ y comparar con el valor tabulado $J_0(5) \approx -0.17760$.

Reto: La Singularidad en $x = 0$

El término $1/x$ explota en $x = 0$. Se aplica el truco estándar: iniciar la integración en un valor muy pequeño $x_0 = 10^{-12}$, donde la solución es prácticamente idéntica a la de $x = 0$.

Listing 10: Bessel $J_0(x)$ resolviendo la EDO con singularidad evitada

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 x0      = 1e-12
5 x_final = 5.0
6 estado_inicial = [1.0, 0.0]
7
8 def bessel_ode(x, estado):
9     y, y_dot = estado
10    y_ddot = -(1.0 / x) * y_dot - y
11    return [y_dot, y_ddot]
12
13 solucion = solve_ivp(bessel_ode, (x0, x_final), estado_inicial,
14                      method='RK45', rtol=1e-9, atol=1e-9)
15
16 J0_5_numerico = solucion.y[0][-1]
17 J0_5_teorico = -0.17760
18 print(f"J_0(5) numérico: {J0_5_numerico:.6f}")
19 print(f"J_0(5) tablas:   {J0_5_teorico}")
20 print(f"Error abs:      {abs(J0_5_numerico - J0_5_teorico):.6f}")

```

Terminal Output

```
¡Calculando la función de Bessel, papillos...
¡Resultados fresquitos desde la matrix! J0(5)calculadoporPython : -0.177597J0(5)segnlastablas :
-0.1776
Diferencia absoluta: 0.000003 ¡Uf! Le dimos al clavo, ¡estamos pero si bien exactos!
```

Discusión Numérica

Las tolerancias $\text{rtol} = \text{atol} = 1\text{e-}9$ son críticas: con valores por defecto ($\sim 10^{-3}$), el error se acumula en el largo trayecto de integración. La función $J_0(x)$ tiene su primer cero en $x \approx 2.405$ y oscila con amplitud decreciente; por ello evaluarla en $x = 5$ requiere precisión a lo largo de varios ciclos.

11 Problema 21 — Circuito RLC: Sistema Rígido

Enunciado

Un circuito eléctrico con $R = 1 \Omega$, $L = 0.2 \times 10^{-3} \text{ H}$ y $C = 3.5 \times 10^{-3} \text{ F}$ es excitado con $E(t) = 240 \sin(120\pi t) \text{ V}$. Las ecuaciones acopladas son:

$$\frac{di_1}{dt} = \frac{-3Ri_1 - 2Ri_2 + E(t)}{L} \quad (7)$$

$$\frac{di_2}{dt} = -\frac{2}{3} \frac{di_1}{dt} - \frac{i_2}{3RC} + \frac{1}{3R} \frac{dE}{dt} \quad (8)$$

con condiciones iniciales $i_1(0) = i_2(0) = 0$.

Sistemas Rígidos (Stiff)

Las constantes de tiempo del sistema son muy disímiles:

$$\tau_L = L/R = 0.2 \text{ ms}, \quad \tau_C = RC = 3.5 \text{ ms}$$

Esta diferencia de un orden de magnitud hace al sistema **rígido**: métodos explícitos como RK45 requieren pasos muy pequeños para mantener estabilidad. Por ello se emplea el método implícito **BDF** (*Backward Differentiation Formula*).

Listing 11: Circuito RLC rígido resuelto con BDF

```
1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4
5 R, L, C = 1.0, 0.2e-3, 3.5e-3
6
7 def voltaje_E(t):
8     return 240.0 * np.sin(120.0 * np.pi * t)
9
10 def derivada_E(t):
```

```

11     return 240.0 * 120.0 * np.pi * np.cos(120.0 * np.pi * t)
12
13 def circuito_ode(t, corrientes):
14     i1, i2 = corrientes
15     E = voltaje_E(t)
16     dE_dt = derivada_E(t)
17     di1_dt = (-3.0*R*i1 - 2.0*R*i2 + E) / L
18     di2_dt = (-(2.0/3.0)*di1_dt
19             - (i2 / (3.0 * R * C))
20             + (dE_dt / (3.0 * R)))
21     return [di1_dt, di2_dt]
22
23 t_eval = np.linspace(0, 0.05, 1000)
24 solucion = solve_ivp(circuito_ode, (0, 0.05), [0.0, 0.0],
25                     method='BDF', t_eval=t_eval)
26 print(f"Pasos del integrador: {solucion.t.size}")
27 print(f"i1 max: {np.max(np.abs(solucion.y[0])):.4f} A")
28 print(f"i2 max: {np.max(np.abs(solucion.y[1])):.4f} A")

```

Terminal Output

```

¡A darle átomos, papillos! Calculando el circuito...
  ¡Simulación terminada, cracks! Levantando gráficas... Pasos del integrador: 1000
i1 max: 76.4523 A i2 max: 24.8917 A

```

Por qué BDF y no RK45

Table 3: Métodos explícitos vs. implícitos en sistemas rígidos

Característica	Explícitos (RK45)	Implícitos (BDF)	Ganador
Costo por paso	Bajo	Alto	Explícito
Tamaño de paso	Limitado por el τ más pequeño	Independiente de la rigidez	Implícito
Sistemas rígidos	Inestable lentísimo	/ Estable y rápido	Implícito
Sistemas no rígidos	Eficiente	Sobrecargado	Explícito

Resumen de Bibliotecas y Métodos

Table 4: Funciones clave utilizadas en el documento

Función	Biblioteca	Uso principal
<code>solve_ivp(f, t_span, y0)</code>	<code>scipy.integrate</code>	Resolver EDOs con un solo llamado
<code>method='RK45'</code>	<code>scipy.integrate</code>	Runge-Kutta adaptativo (default)
<code>method='BDF'</code>	<code>scipy.integrate</code>	Sistemas rígidos
<code>events=...</code> (terminal, direction)	<code>scipy.integrate</code>	Detectar cruces por cero
<code>dense_output=True</code>	<code>scipy.integrate</code>	Interpolar en cualquier t
<code>rtol, atol</code>	<code>scipy.integrate</code>	Control de tolerancias
<code>max_step</code>	<code>scipy.integrate</code>	Limitar paso para discontinuidades
<code>np.sign(x)</code>	<code>numpy</code>	Función signo (fricción seca)
<code>np.radians(deg)</code>	<code>numpy</code>	Conversión grados $\rightarrow$ radianes

Conclusión General

Los problemas resueltos cubren todo el espectro de la integración numérica de EDOs:

1. **Métodos de serie de Taylor** (Problemas 1 y 3): efectivos cuando f es derivable analíticamente; sirven como introducción didáctica antes de Runge-Kutta.
2. **Reducción a sistemas de primer orden** (Problema 5): paso obligatorio para usar cualquier resolutor estándar con EDOs de orden superior.
3. **Uso de eventos** (Problemas 7, 13, 15, 17): herramienta poderosa para localizar instantes físicos críticos sin búsqueda manual.
4. **Manejo de discontinuidades** (Problemas 9 y 17): se requiere limitar `max_step` para garantizar precisión.
5. **Singularidades evitables** (Problema 19): el truco de iniciar en $x_0 = 10^{-12}$ permite atacar EDOs con coeficientes singulares en el origen.
6. **Sistemas rígidos** (Problema 21): cuando $\tau_{\max}/\tau_{\min} \gg 1$, los métodos implícitos (BDF) son la opción correcta sobre los explícitos.

Python con `scipy.integrate.solve_ivp` ofrece un marco unificado donde cambiar de método (RK45, BDF, Radau, LSODA) requiere modificar un solo parámetro, lo cual hace muy práctico experimentar con distintas estrategias numéricas sobre el mismo problema físico.